Clase 06: Paralelismo

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Martes 24 de Marzo del 2026

Contenidos

Modelos de memoria y programación paralela

Tipos de paralelismo y granularidad

Ejemplos de algoritmos paralelos

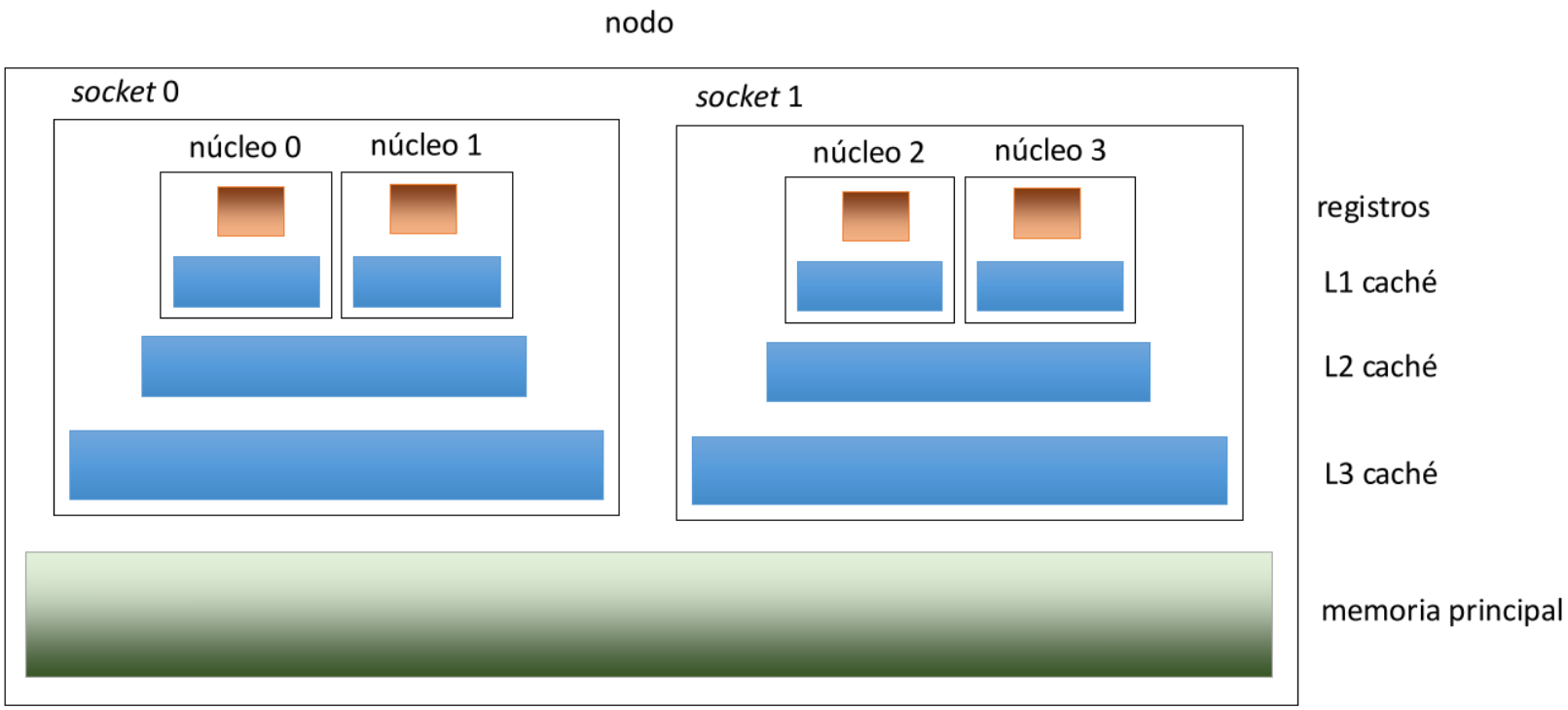
Métricas de rendimiento paralelo

Paralelismo

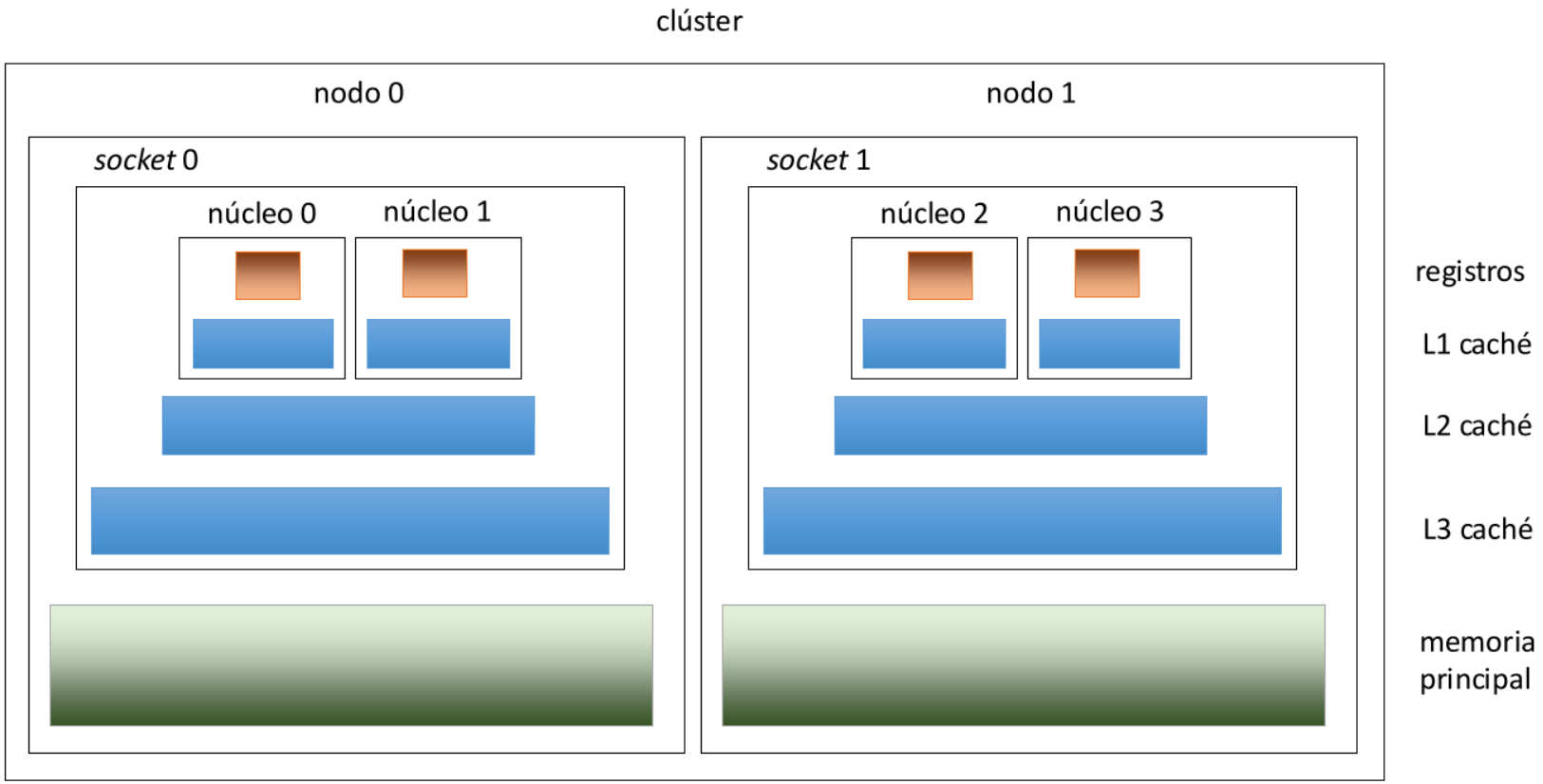
Computador paralelo: posee más de una unidad de cómputo para trabajar en el mismo problema.

- Si hay un solo núcleo, ILP (Instruction Level Parallelism)
- ILP: determinado por arquitectura y compilador.
- En general: más de un flujo de instrucciones manejado por distintos núcleos.
- Esto se puede programar explícitamente.

Los procesadores *multi-core*: memoria compartida



Los clústers: memoria distribuida



Modelos de programación paralela

Memoria compartida

- Los hilos comparten un espacio de direcciones (OpenMP).
- Comunicación implícita a través de variables compartidas.

Memoria distribuida

- Cada proceso tiene su propia memoria; se comunican por mensajes (MPI).
- Comunicación explícita: el programador controla qué se envía y cuándo.

Híbrido

- Combinación de ambos modelos (MPI + OpenMP).

Tipos de Paralelismo

Data parallelism

- La misma operación aplicada en paralelo a muchos datos.
- Común en vectores, matrices, grafos.

Functional parallelism

- Instrucciones distintas dentro de un algoritmo se envían a unidades de cómputo distintas.
- ILP es un caso particular. *Pipelining*.

Task parallelism

- Trabajos independientes se pueden realizar de manera simultánea.
- Paradigma de *multiprocessing*.

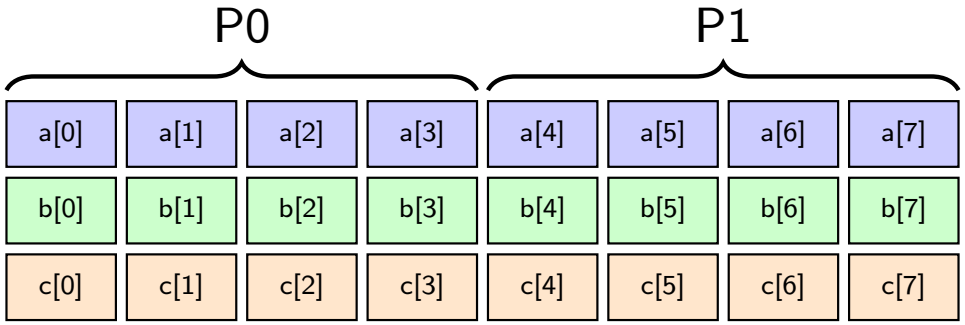
Data Parallelism

Ejemplo: suma de vectores

Secuencial:

```
for(i = 0; i < n; i++)
    c[i] = a[i] + b[i];
```

Paralelo: cada procesador calcula una porción independiente.

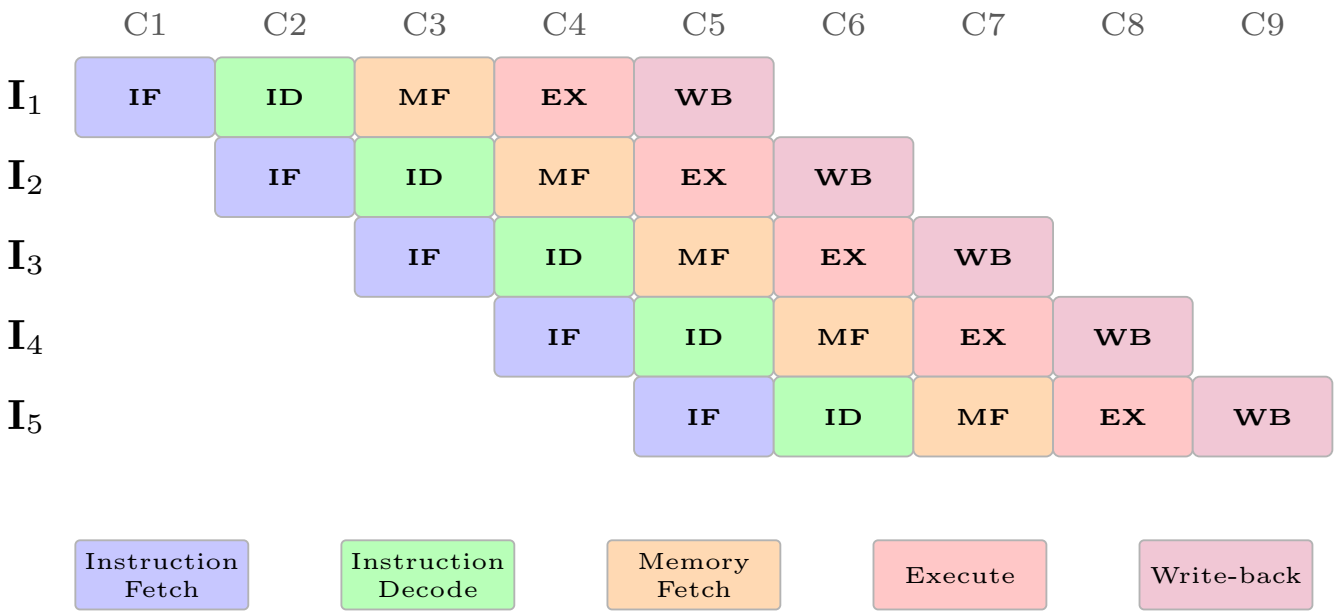


- Cada elemento es independiente $\Rightarrow$ paralelismo perfecto.

Functional Parallelism

Ejemplo: ILP / Pipelining

Distintas instrucciones en distintas etapas del procesador al mismo tiempo.

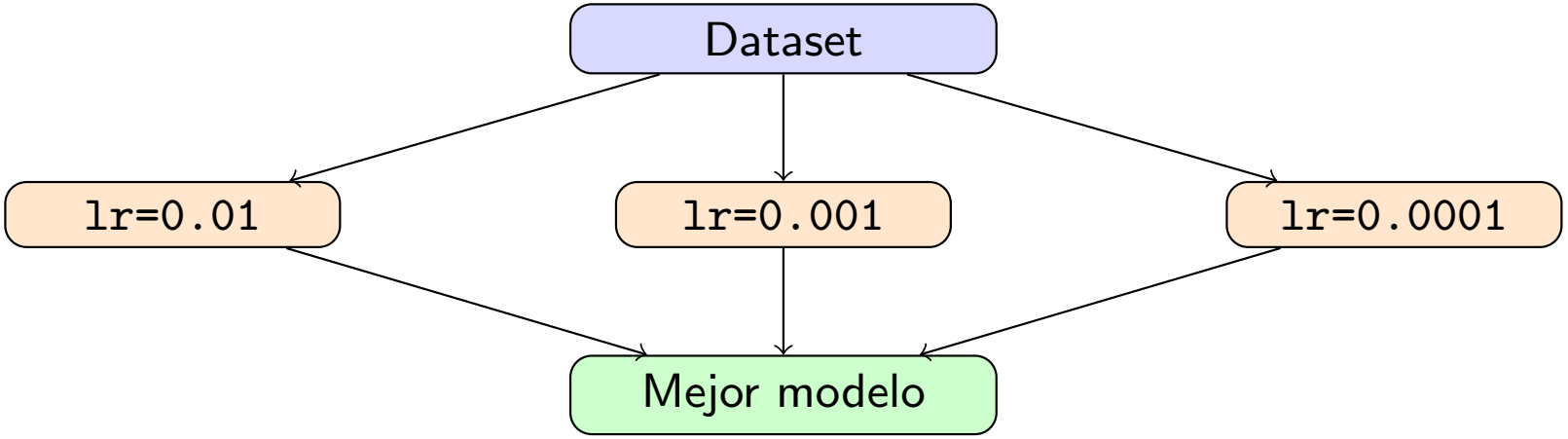


- Las etapas (IF, ID, MF, EX, WB) se solapan entre instrucciones.

Task Parallelism

Ejemplo: Parameter Tuning en ML

Entrenar un modelo con distintas configuraciones de hiperparámetros de forma simultánea.



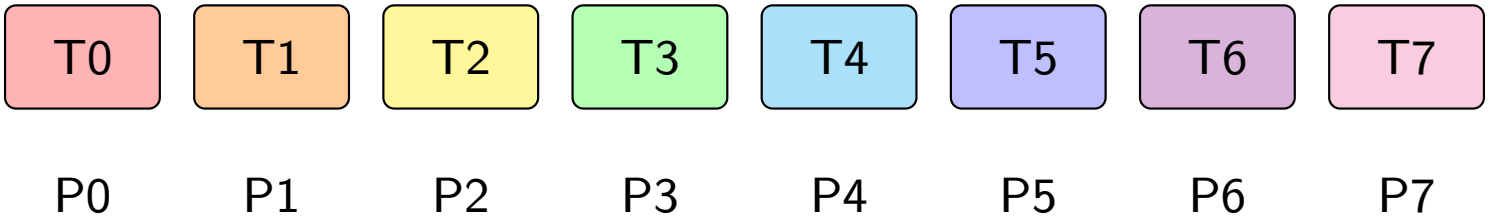
- Cada tarea es independiente : distintos hiperparámetros, mismos datos.

Granularidad

- **Grano fino:** muchas tareas pequeñas; mayor potencial de paralelismo pero mayor *overhead* de sincronización.
- **Grano grueso:** pocas tareas grandes; menor *overhead* pero puede dejar recursos ociosos.
- La granularidad óptima depende del problema, la arquitectura y el costo de comunicación.

Granularidad

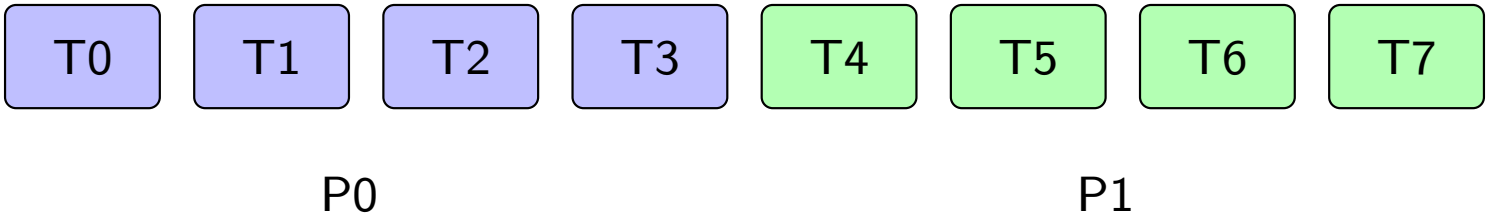
Grano fino



- Ejemplo: cada hilo suma un único elemento de un vector.
- Muchas tareas pequeñas $\rightarrow$ mayor *overhead* de sincronización.

Granularidad

Grano grueso



- Ejemplo: cada procesador trabaja sobre la mitad de un dataset.
- ¿Qué pasa si P0 termina antes que P1? → P0 queda ocioso.

Paralelismo

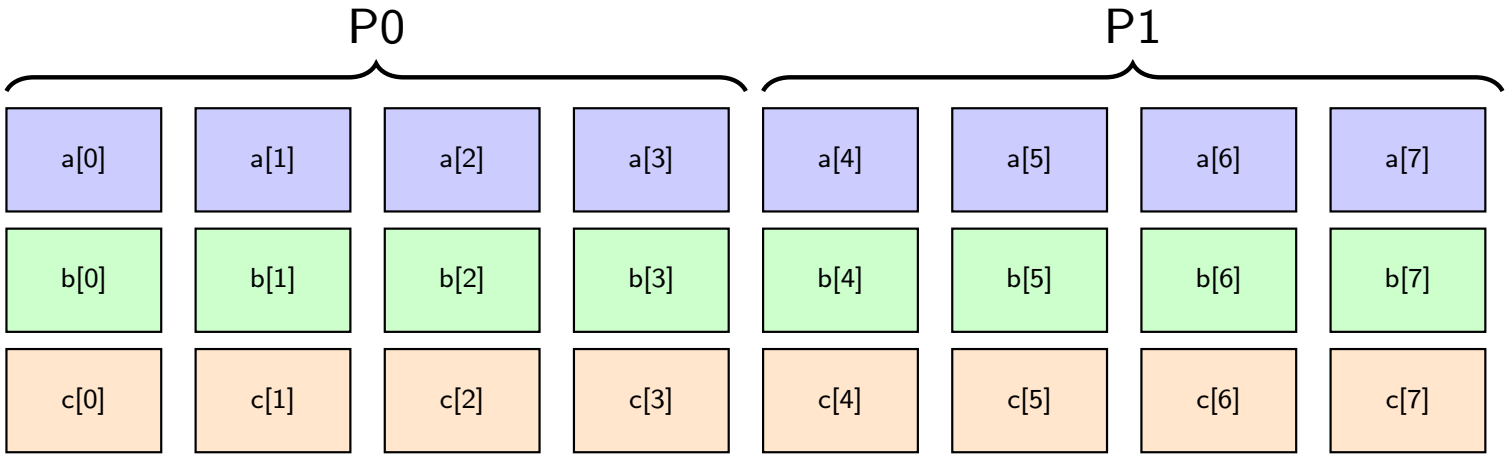
En muchas ocasiones, el trabajo corresponde a la misma operación aplicada a muchos datos.

¿Cuánto puede ser procesado al mismo tiempo por distntas unidades de cómputo?

- n operaciones a ejecutar.
- Un procesador toma tiempo t.
- Si hay p procesadores, ¿pueden las n operaciones ser procesadas en tiempo $\frac{t}{p}$?

Paralelismo

```
1   for(i = 0; i < n; i++){
2      a[i] = b[i] + c[i];
3   }
```



Paralelismo

```
1    for(i = 0; i < n; i++){
2       a[i] = b[i] + c[i];
3    }
```

- Algoritmo original en un procesador toma tiempo n.
- Algoritmo con p procesadores toma tiempo $\frac{n}{p}$.
- Notar que esto asume que todos los procesadores tienen acceso a memoria compartida.
- ¿Cuánto más rápido es el algoritmo paralelizado respecto al original?

Paralelismo

Operación de reducción:

```
1        s = 0;
2        for(i = 0; i < n; i++){
3           s += x[i];
4        }
```

¿Es esto paralelizable?

- Existen dependencias entre las operaciones.
- En memoria compartida: posible condición de carrera.
- ¿Cómo se pueden dividir tareas para alcanzar paralelismo?
- Es necesario modificar el algoritmo.

Paralelismo

Cómputos locales

```
1        s = 0;
2        for(i = 0; i < n; i++){
3           s += x[i];
4        }
```

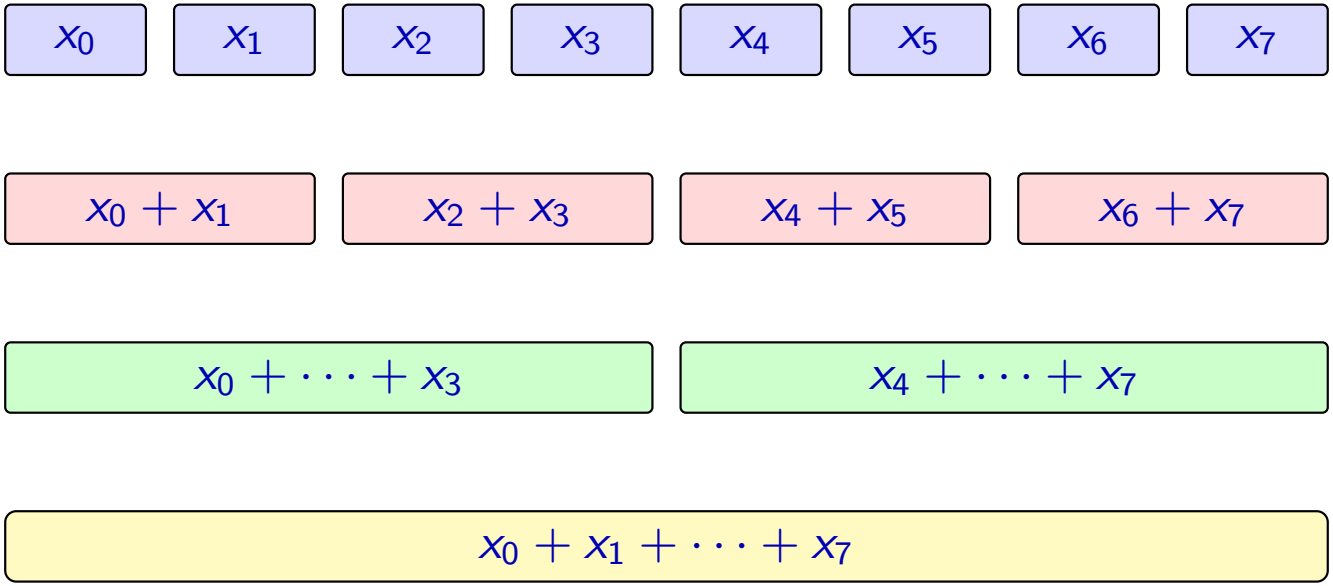
- Se divide el arreglo de largo n en p procesadores.
- Cada procesador recibe un arreglo de largo n/p y calcula una suma local.
- Posteriormente, los resultados locales se suman para lograr la reducción completa.
- Se realizó paralelismo sobre los datos del problema.

Paralelismo

Operación de reducción:

```
s = 0;
for(i = 0; i<n; i++){
   s += x[i];
}
```

¿Es esto paralelizable? $\rightarrow$ *In-place reduction*



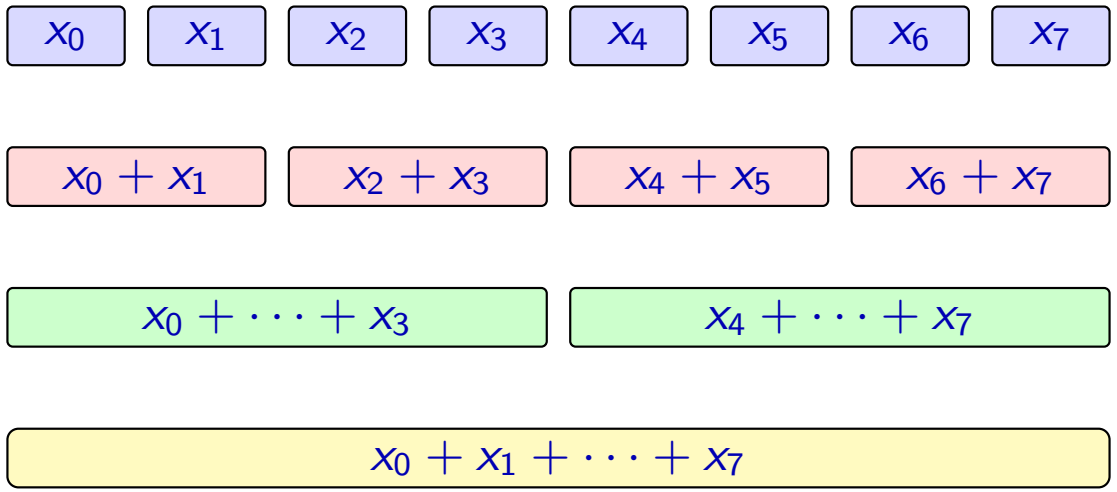
Paralelismo

```
1    for(s=2; s<2*n; s*=2){
2       for(i = 0; i<n-s/2; i+=s){
3          x[i] += x[i+s/2];
4       }
5    }
```

- El `for` exterior ejecuta $\log_2 n$ iteraciones.
- El `for` interno se puede distribuir entre p procesadores.
- ¿Cuánto más rápido es el algoritmo paralelizado respecto al original?
- Si p es suficientemente grande, el tiempo es $\mathcal{O}(\log_2 n)$.

Paralelismo

```
1    for(s=2; s<2*n; s*=2){
2       for(i = 0; i<n-s/2; i+=s){
3          x[i] += x[i+s/2];
4       }
5    }
```



- Memoria compartida, se pueden usar las mismas direcciones.
- Memoria distribuida, deben transmitirse los datos que necesitan.

Speedup

Compara el tiempo con un procesador, versus el tiempo con múltiples procesadores.

- T_1 : Tiempo de ejecución con 1 unidad de cómputo.
- T_p : Tiempo de ejecución con p unidades de cómputo.

El **Speedup** es

$$S_p = \frac{T_1}{T_p}$$

Eficiencia

Al evaluar los tiempos de ejecución en paralelo

- Caso ideal: tiempo con p procesadores es $T_p = \frac{T_1}{p}$, es decir $S_p = p$.
- En la práctica tenemos $S_p \leq p$.

Llamamos **Eficiencia** a la métrica

$$E_p = \frac{S_p}{p}.$$

Si $S_p \leq p$, entonces $0 < E_p \leq 1$.

Costo

Si el *speedup* no es perfecto, se puede definir el *overhead* como

$$T_0 = pT_p - T_1$$

Puede ocurrir

- El algoritmo paralelo puede no ser el mismo que el algoritmo secuencial.
- El algoritmo paralelo puede requerir enviar mensajes de coordinación previo al cómputo.

El algoritmo es *cost-optimal* si el *overhead* es del orden de ejecución del algoritmo secuencial.

¿Qué limita el *speedup*?

- Parte secuencial: toda fracción del código que no puede paralelizarse impone un techo al speedup.
- Comunicación y sincronización: coordinar múltiples unidades de cómputo tiene un costo que crece con p.
- Desbalance de carga: si un procesador trabaja más que los demás, los demás lo esperan inactivos.

Ley de Amdahl

- No todo el código es paralelizable.
- Existe una fracción secuencial f_s y una fracción paralelizable f_p con

$$f_s + f_p = 1$$

- El tiempo con p procesadores es

$$T_p = T_1 \left(f_s + \frac{f_p}{p} \right)$$

- El Speedup es

$$S_p = \frac{T_1}{T_p} = \frac{1}{f_s + \frac{f_p}{p}}$$

- La Eficiencia es

$$E_p = \frac{1}{pf_s + f_p}$$

Ley de Amdahl

- El Speedup es

$$S_p = \frac{T_1}{T_p} = \frac{1}{f_s + \frac{f_p}{p}}$$

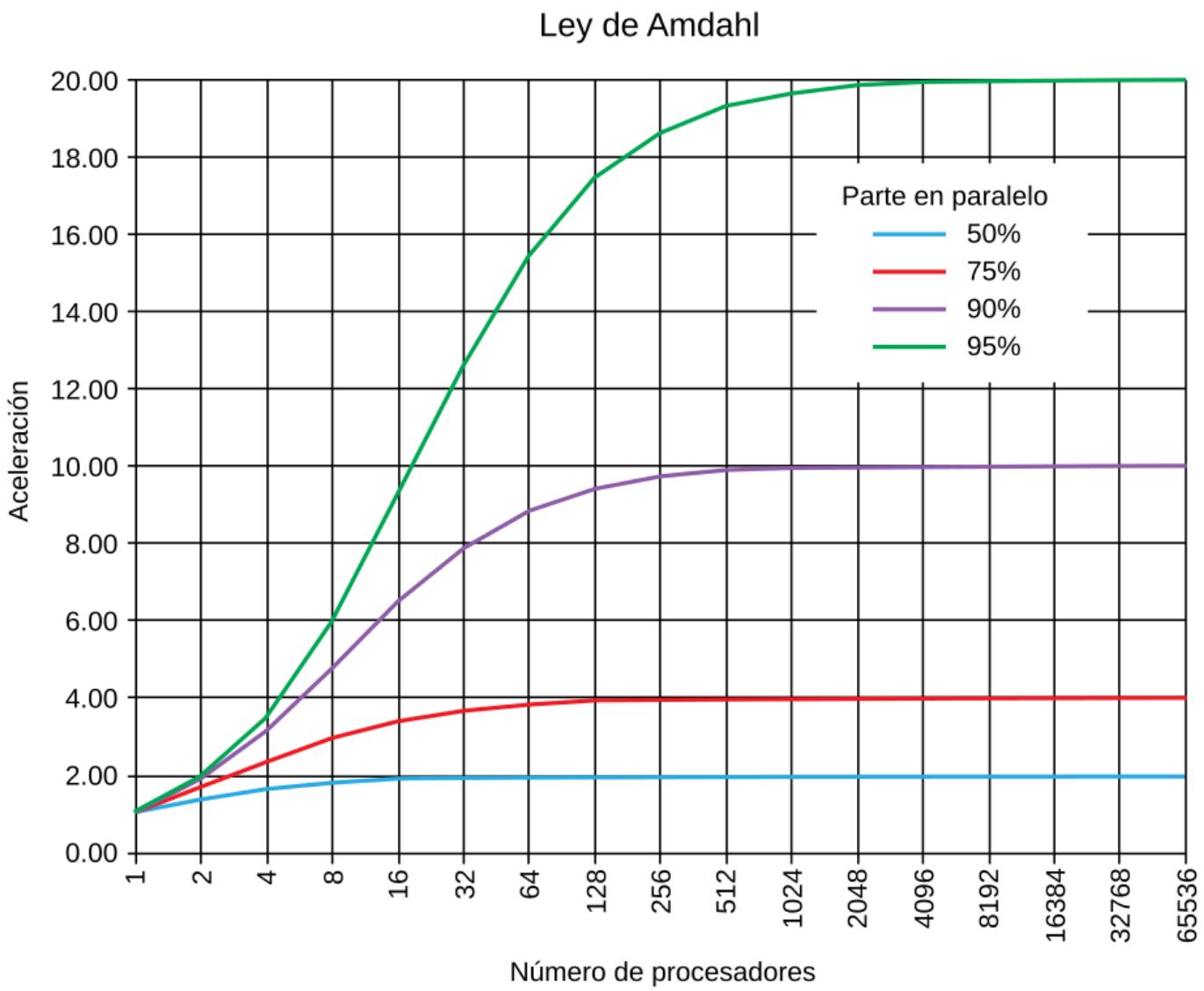
- La Eficiencia es

$$E_p = \frac{1}{pf_s + f_p}$$

Cuando $p \to \infty$ tenemos

$$S_p \leq \frac{1}{f_s}, \qquad E_p \sim \frac{1}{p}$$

Ley de Amdahl



Ley de Amdahl

- Para un problema de tamaño fijo, existe un punto en que ya no vale la pena aumentar el número de procesadores.
- En el gráfico anterior, agregar más procesadores no suma ventajas.
- En la práctica, puede incluso empeorar: tiempo de comunicación y *setup*.
- No es necesario usar todos los recursos que tenemos para un problema fijo.

Ley de Gustafson

- Amdahl: fijo el trabajo (N constante) y aumento $p \Rightarrow$ speedup acotado.
- Gustafson: fijo el tiempo de ejecución paralelo T_p y aumento N con p.
- Pregunta relevante: si tengo p veces más cómputo, ¿cuánto más trabajo puedo resolver en el *mismo* tiempo?

Ley de Gustafson

Sea T_p el tiempo de ejecución con p procesadores (lo que medimos).

- f_s: fracción de *ese tiempo* dedicada a la parte secuencial. $T^{sec} = f_s \cdot T_p$
- f_p: fracción de *ese tiempo* dedicada a la parte paralela. $T^{par} = f_p \cdot T_p$
- Con $f_s + f_p = 1$.

Si ese mismo trabajo lo hiciera un solo procesador, la parte paralela tardaría p veces más:

$$T_1 = f_s T_p + p f_p T_p$$

Ley de Gustafson

- La ley de Amdahl asume una cantidad fija de trabajo.
- A mayor nivel de cómputo, es razonable aumentar la cantidad de trabajo.
- El tiempo de cómputo con p procesadores será

$$T_p = T_p(f_s + f_p) = T^{sec} + T^{par}$$

- El Speedup será

$$S_p = \frac{T_1}{T_p} = \frac{T_p f_s + p T_p f_p}{T_p(f_s + f_p)} = p - (p-1) f_s$$

Comparación

- Con la ley de Amdahl tenemos

$$S_p = \mathcal{O}(1),$$

es decir, la eficiencia paralela es

$$E_p = \mathcal{O}(1/p).$$

- Con la ley de Gustafson tenemos

$$S_p = \mathcal{O}(p),$$

es decir, la eficiencia paralela es

$$E_p = \mathcal{O}(1).$$

- Equilibrar el trabajo entre procesadores aprovecha mejor los recursos.

Escalabilidad

- **Escalabilidad fuerte:** para un problema fijo, el tiempo de ejecución disminuye linealmente con el número de procesadores.

  $N \equiv \mathrm{const},\ p \to \infty$, entonces se tiene $E_p \approx \mathrm{const}.$

- **Escalabilidad débil:** el tamaño y número de procesadores crece de manera que el trabajo por procesador se mantiene constante..

  $N \to \infty,\ p \to \infty,\ M = N/p \equiv \mathrm{const}$, entonces se tiene $E_p \approx \mathrm{const}.$